DEEP LEARNING LAB ASSIGNMENT 3

NAME :- SOHA SAMEER SAYYED

PRN:- 202301100023

BATCH:- DEEP LEARNING 4

TOPIC :- Design and implement a multi-task computer vision system using **modern YOLO models (v8 and beyond)** to perform:

- Object Detection
- Image Classification
- Pose Estimation
- Oriented Bounding Boxes (OBB)
- Deployment using a nano/lightweight model on localhost

1. This project focuses on building a multi-task computer vision system using the Ultralytics YOLOv8 framework. The system is capable of performing multiple vision tasks such as object detection, image classification, pose estimation, and oriented bounding box (OBB) detection.

The main objective of this project is to understand how a single deep learning framework like YOLO can be applied to solve different computer vision problems efficiently. All tasks were implemented and executed locally without the use of any cloud-based services.

2. Objectives

• To implement object detection using YOLOv8
• To perform image classification using YOLO classification models
• To detect human keypoints using pose estimation
• To implement oriented bounding box (OBB) detection
• To deploy the trained model locally using webcam

3. Tools and Technologies Used

• Python
• Ultralytics YOLOv8
• VS Code
• Roboflow (for dataset collection)

4. Datasets Used

Object Detection Dataset

Car Detection Dataset
https://universe.roboflow.com/drink-tracker/car-detection-pn2gk

Image Classification Dataset

Dog vs Cat Dataset
https://universe.roboflow.com/tae-7jslq/dog-vs-cat-5fmu0

Pose Estimation Dataset

COCO Pose Dataset (used via Ultralytics built-in configuration)
https://docs.ultralytics.com/datasets/pose/

Oriented Bounding Box Dataset

Rotated Object Dataset
https://universe.roboflow.com/dartcounter/rotated-idn1l/dataset/1

5. Methodology

Step 1: Environment Setup

A Python virtual environment was created and required libraries such as Ultralytics YOLO and OpenCV were installed using pip. The entire implementation was done locally using VS Code.

Step 2: Data Preparation

Datasets were downloaded from Roboflow and extracted into the project directory. The datasets were organized according to YOLO requirements:

- Detection and OBB datasets used YAML configuration files
- Classification dataset used folder-based structure (train/valid/test)
- Pose dataset was loaded using built-in YOLO configuration

Step 3: Model Training

Object Detection
A YOLOv8 nano model was trained on the car detection dataset to detect objects using bounding boxes.

Image Classification
A classification model was trained to classify images into categories such as cats and dogs.

Pose Estimation
A YOLOv8 pose model was trained to detect human keypoints such as joints and body structure.

Oriented Bounding Box (OBB)
A YOLOv8 OBB model was trained using a rotated object dataset to detect objects with orientation.

Step 4: Model Evaluation

After training, all models were evaluated using validation datasets. The outputs obtained were:

- Bounding boxes for object detection
- Class labels for classification

• Keypoints and skeleton structure for pose estimation
• Rotated bounding boxes for OBB

Step 5: Local Deployment

A local deployment was implemented using OpenCV and YOLO. A Python script was created to access the webcam and perform real-time detection using the trained model.

6. Conclusion

This project successfully demonstrates the implementation of a multi-task computer vision system using YOLOv8. It highlights the flexibility of YOLO in handling different types of vision tasks such as detection, classification, pose estimation, and oriented bounding box detection.

All tasks were implemented locally, fulfilling the assignment requirements. The project also provided practical exposure to dataset handling, model training, debugging, and deployment.

GitHub Repository Link:

https://github.com/SohaSayyed/Deep-Learning-Practical-no-03-YOLO-multitask-Project